Scorer in a Box: A One-GPU Replica of the Gemma 4 Agent Scorer, and the Silent Failures It Catches

An open kit for evaluating, debugging and fine-tuning local coding agents under the competition's real conditions

Shashanka B R[1]
[1]IBM
shashanka.b.r@ibm.com

Gemma 4 Developer Agent Paper Track, Best New Resource
Code: https://github.com/shashanka300/runtime-bound

November 2026

Abstract

Every team in this competition develops against a scorer it cannot see. Submissions take a day to score, each team gets one a day, and the score comes back as a single number with no traces. So when a change helps locally and hurts on the board, there is no way to tell whether the agent got worse or the local setup was lying. We built a kit that answers that question on one consumer GPU. It serves the scorer's exact engine build with the scorer's flags, runs the official harness at the scorer's speed, compiles bundles with the scorer's own code before they spend a daily slot, and probes the parts of the stack that fail silently: whether an adapter is applied, whether the thinking budget reaches the engine, and whether the engine renders multi-turn history the way the agent assumes. Around that core sit trace analyses that show where an agent's time goes, a pipeline that turns 116 outside repositories into competition-format tasks, and a LoRA trainer for the competition's int4 checkpoint that standard tooling cannot fine-tune. In seven weeks the kit caught six silent failures, three in the scorer itself (all confirmed or fixed by the organizers) and three in our own setup that looked exactly like real results. We describe each component, the failure it exists to catch, and how to use it.

1 Introduction

Agent competitions evaluate a whole stack, not a model: a serving engine, a chat template, an agent framework, a sandbox, a dependency set and a clock. In the Gemma 4 Developer Agent competition [7] that stack runs on 4×L4 GPUs behind a submission queue. Participants see a public development set of 129 tasks and a leaderboard number per day.

That combination makes silent failures expensive. A wiped adapter, an ignored parameter or a broken test environment does not raise an error; it changes the score by a few points, and a few points is inside the board's noise. Our first submission solved 31% of tasks locally and scored 0.03 on the board; the companion paper [9] traces that gap. This paper describes the tooling that made the trace possible, released as a resource.

The kit is built around one rule: *when local and scorer can differ, run the scorer's own code.* Where we could not run it, we built a probe that tells the two apart. Table 1 lists what that caught.

Component	Silent failure it caught	Outcome
LoRA canary	The scorer's vLLM build wiped adapter weights: a random-weight canary gave output identical to the base model.	Fixed by the organizers.
Thinking cross-check	The harness validated `thinking_budget` but never sent it to the engine.	Fixed (forum 744566).
Record + replay proxy	vLLM 0.19.1 dropped the agent's earlier `reasoning_content` from the prompt (418 tokens shorter at turns 2–4, 2,583 from turn 5).	Fixed; independently reported (forum 744354).
Serving-flag audit	vLLM's YAML loader silently drops boolean `false`; a KV-reservation flag was never applied, causing stalls we first read as a 12.5-point engine effect.	Local; flag now passed on the command line.
Per-pool wheel cache	The harness caches unpacked wheels under one fixed name; a cache from another wheel pool broke pytest on every task of a screen (0/19).	Local fix; reported (forum 744029).
Gold-patch sweep	Only 41 of 129 public tasks can be graded offline with the released environment.	Reported; hidden set validated at 100% by the organizers.

Table 1: Six silent failures and the component that caught each. None of them raised an error.

2 The kit

Figure 1 shows how the parts connect. Everything runs on one RTX 5090 (32 GB) under WSL2; the LoRA trainer also runs on a rented H100.

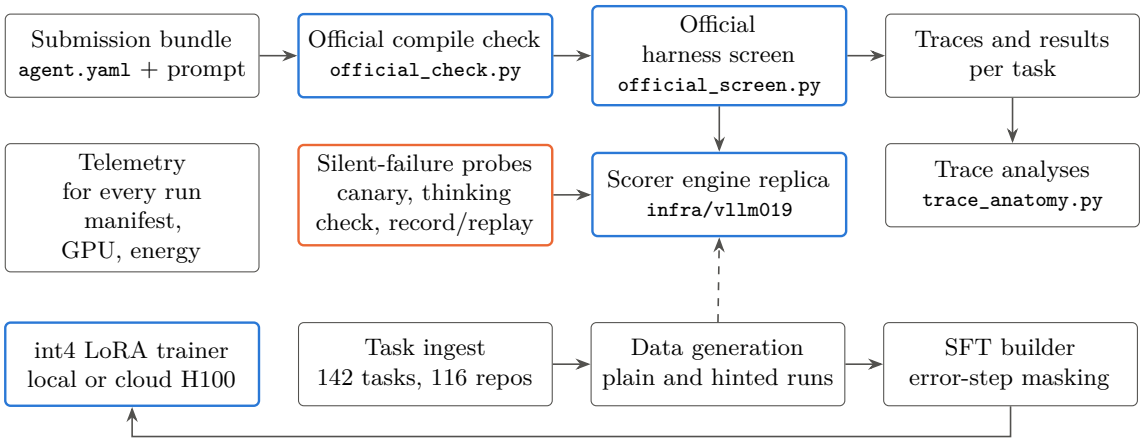


Figure 1: The kit. Blue: parts that run the scorer's own code or reproduce its configuration. Orange: probes that detect where local and scorer behaviour silently diverge. The training path (bottom) reuses the same engine and harness to generate data.

2.1 Scorer engine replica

`infra/vllm019/` installs the scorer's vLLM 0.19.1 [6] and every serving dependency from the organizers' wheelhouse with `-no-deps` at the pinned versions, so a resolver cannot pull a newer transformers (it did, once). It serves with the scorer's flags and passes boolean flags on the command line, because the YAML config path drops `false`. A 12B profile adds the architecture override the organizers published for that checkpoint.

2.2 Official harness screen

`tools/official_screen.py` evaluates a bundle with the official harness (swegemma, adk_submission, google-adk at the wheelhouse versions), configured like the organizers' getting-started notebook, against the local engine. Two choices make its numbers comparable to the scorer: tasks run one at a time, as on the scorer, and the per-task time cap is scaled by 0.625 because the L4 box decodes about 1.6× slower. It gives each wheel pool its own temporary directory, since the harness's wheel cache is keyed by a fixed name. `tools/overnight_screen.sh` wraps it with engine start-up, a health watchdog and resumable rows.

2.3 Compile check before submitting

`leaderboard/official_check.py` validates and compiles a bundle with the scorer's own submission code and the real tool signatures, without calling a model. With one submission per day and a day-long queue, a packaging error costs a day, so we run it on every bundle before submitting. `leaderboard/build.py` generates every bundle from code, so each submitted file has a recorded derivation.

2.4 Silent-failure probes

- **LoRA canary** (`tools/lora_canary*.py`) copies an adapter's layout, fills `lora_B` with random values and compares greedy output and top-5 log-probabilities against the base model. An applied adapter must change them; on the original scorer build it did not, on the patched build the canary's log-probability moved from -13.125 to -12.19. An all-linear variant also measures the decode cost of an adapter (66.9 to 57.4 tokens/s, -14%, for rank 16 on all linear layers).
- **Thinking cross-check** (`tools/check_official_thinking.py`) runs a grid of sampling configs through the scorer's own translation function and exits non-zero if the local harness would send anything different. It is meant to run after every wheelhouse update.
- **Record and replay** (`tools/record_proxy.py`, `tools/replay_requests.py`) is a transparent proxy that records every chat request of a real agent session, and a replayer that sends the identical requests to another engine. Because inputs are fixed, any difference in prompt length or output is the engine's rendering of history, not the agent.

2.5 Gradability audit

`tools/official_gold_sweep.py` applies each task's reference patch and runs its tests through the official air-gapped verification path. It found that only 41 of 129 public tasks are gradable offline with the released environment (rich 39/48, httpx 1/1, fastapi 1/67, requests 0/13), with the causes per task (missing test packages, SSL failures on Python 3.13, installed packages shadowing the workspace). The result, `leaderboard/official_gradability.csv`, is the task list every screen uses. Our own harness, swebox, fills missing packages at image build and grades 114 of 129; `tools/regrade_swebox.py` uses it to grade runs the official grader cannot (it matched the official grader on 4 of 4 overlapping patches).

2.6 Trace analyses

`tools/trace_anatomy.py` turns a screen's traces into a per-task anatomy: turns, output tokens, the longest turn, the share of time in the three slowest turns, time to the first edit, and repetition loops

(turns where more than a quarter of the sentences repeat). It produced the finding that timed-out tasks lose 62% of their time in one to three turns at the thinking cap [9]. `tools/behaviour_scan.py` classifies every tool call (duplicate, overlapping re-read, repeated search, error, retry) to measure wasted calls.

2.7 Telemetry

Every run writes a manifest (git state, config hash, package and lockfile hashes, GPU, driver, engine version, served model, data fingerprint), structured logs, spans, 1 Hz NVML samples with the energy counter, container resource use and the engine's Prometheus counters. `rbcore.windows` attributes any of them to a task or phase, for example energy per task. `tools/otel_bridge.py` forwards training and cost metrics to any OTLP dashboard; we used it to watch a cloud training run against a credit cap. The KV-reservation artefact in Table 1 was found in this telemetry, as requests waiting with an empty KV cache.

2.8 Multi-repository tasks

The public set covers four repositories, and the hidden set others. `tools/ingest_rebench.py` converts SWE-rebench [2] tasks into the competition's on-disk format: task records, a snapshot of each repository at its base commit, Python 3.13 wheels for its dependencies, and empty graph and embedding folders. A self-check keeps only tasks whose reference patch passes and whose empty patch fails. From 319 snapshotted tasks it kept **142 tasks from 116 repositories**; the 175 rejects are recorded with their failure class (collection errors 133, failing tests 41). Because the format matches, the official harness and every tool above run on them unchanged. We release the script, not the data; the source dataset is CC-BY-4.0.

2.9 Fine-tuning the int4 checkpoint

The competition's 31B checkpoint stores its linear layers in a packed int4 format (compressed-tensors, group size 32) that standard PEFT tooling cannot wrap. `tools/qlora.py` replaces each of the 410 packed layers with a module that unpacks its weight to bf16 inside forward and backward (bit-exact against the reference unpacker, nothing saved for backward) and adds LoRA [3, 4]. `tools/train_lora.py` trains on overlapping 8k-token windows with gradient checkpointing, keeps the optimizer and master weights on the CPU so a 32 GB card fits rank 16 on all linear layers (122.4M parameters), masks the loss on tool calls that failed as calls, holds out validation runs, and saves resumable state every ten steps. `tools/build_sft.py` builds the training set from solved runs, including hinted runs with the hint redacted. `tools/modal_train.py` runs the same trainer on a cloud H100 as a detached job that survives a dropped client, with a credit guard. The two machines produced the same initial loss, validation loss and first-step loss (0.2452, 0.3829, 0.3915), so cloud and local runs are interchangeable.

3 Using the kit

A typical loop for a new agent idea is: build the bundle with `leaderboard/build.py`; check it with `official_check.py`; screen it with `overnight_screen.sh` on the 39 gradable tasks (about three hours) or on a multi-repository task root via `COMP_DIR`; compare it paired against the baseline; and read `trace_anatomy.py` before reading the score, because the score alone does not say why. Before trusting any result that involves adapters, sampling or a new wheelhouse, run the canary and the

thinking cross-check. The repository README maps each result in the companion paper to the code and records that produced it.

4 Related work

SWE-bench [5] and its harness define the task and grading format the competition follows; SWE-agent [10] showed how much the agent–computer interface matters for a fixed model, which is why we test interfaces against the scorer's own. SWE-rebench [2] provides continuously collected, decontaminated tasks; our ingest makes them runnable in this competition's environment. SWE-Gym [8] provides executable environments for training agents; our trainer targets the one quantized checkpoint a competitor must use. Paired, interval-based reporting follows Agarwal et al. [1].

5 Limitations

The replica is faithful to the scorer's code, not to its hidden tasks: local solve rates were 3–10× the leaderboard, because the hidden repositories differ from the public ones. The 0.625 time scaling models decode speed only, not prefill. The multi-repository task set has not been validated as a predictor of the leaderboard. The kit targets this competition's stack (vLLM 0.19.1, ADK, swegemma); the probes are general ideas, but their code assumes these interfaces. Several components depend on files only available after accepting the competition rules, which we do not redistribute.

6 Availability

Code, configuration and run summaries: `https://github.com/shashanka300/runtime-bound` (MIT; third-party files listed in `NOTICE`). Every deviation from our pre-registered plan, with the evidence available at the time, is in `docs/design_changes.md`.

References

[1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, 2021.

[2] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2505.20411.

[3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023.

[4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.

[7] Elan Markowitz, Bryan Perozzi, Benedek Rózemberczki, Glenn Cameron, Hadi Hemmati, Yuchen Li, Michael Galkin, Majid Farhadi, Ryan Holbrook, and Ashley Oldacre. Google - the gemma 4 developer agent paper track. https://www.kaggle.com/competitions/gemma-4-developer-agent-paper, 2026. Kaggle.

[8] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-gym. In *International Conference on Machine Learning*, 2025.

[9] Shashanka B R. Runtime-bound: What limits a local 31B coding agent, and what local evaluation gets wrong. Gemma 4 Developer Agent Paper Track, Kaggle, 2026. Companion paper.

[10] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.